

JASA Author Contributions Checklist

Article: A direct approach to tree-guided feature aggregation for high-dimensional regression

Authors: Jinwen Fu, Aaron J. Molstad and Hui Zou

Corresponding author: Aaron J. Molstad, amolstad@umn.edu

Version: 5 October 2026

This checklist describes the code, data and computational outputs in the reproducibility package. The top-level `README.md` gives the directory structure, script-to-result mapping, dependencies and execution order. This preparation version records the available materials and their limitations; complete manuscript reproduction is not certified.

Part 1: Data

☐ This paper does not involve analysis of external data.

☐ I certify that the author(s) of the manuscript have legitimate access to and permission to use the data used in this manuscript.

The access and redistribution certification is not completed in this version.

Abstract

The main-text analyses use processed microbiome feature matrices, outcomes and feature trees from seven studies: Erawijantari 2020, Franzosa 2019, Jacobs 2016, Kim 2020, Sinha 2016, Wang 2020 and Yachida 2019. These support eleven prediction analyses in Table 1. Figure 6 uses the Sinha colorectal-cancer data and taxonomy for data fission and inference. Simulations generate their inputs in code. Processed analysis inputs are in `data/`; source accessions and preprocessing documentation are not included in this version.

Availability

The package contains ten `.RData` files, `sinha_2016_metadata.tsv` and `sinha_2016_genera.tsv` directly under `data/`. The `README` maps these inputs to the analysis scripts. Public-access and redistribution details are not yet documented, so no public or restricted availability category is certified here.

☐ Data **are** publicly available.

☐ Data **cannot be made** publicly available.

Publicly available data

☐ Data are available online.

☐ Data are available as part of the paper's supplementary material.

☐ Data are publicly available by request.

☐ Data are available through another mechanism.

Non-publicly available data

No restricted-data classification or request process is documented. The package does not contain a synthetic or facsimile substitute for the real-data inputs.

Description

File formats

☒ CSV or other plain text: TSV metadata and taxonomy.

☒ Software-specific binary format: R `.RData` files.

☐ Standardized binary format.

☐ Other.

Data dictionary

The prediction runners expect a saved data object containing `x` (feature matrix), `y` (response) and `tree_df` (tree representation). This structural description is not a complete data dictionary. Cohort definitions, variable coding, units, feature identifiers, tree construction and preprocessing are not documented in a separate dictionary in this version.

[] A data dictionary is provided by the authors.

[] The data files are self-describing.

[] A data dictionary is available online.

Part 2: Code

Abstract

The R package `treeFA` implements tree-guided feature aggregation for linear and binary outcomes. `code/core_code/` contains the package, shared helpers and comparison methods. `code/run_code/` separates simulations from real-data analyses. Six simulation runners and eleven prediction runners generate saved results. Additional scripts draw Figures 3-5 and summarize Table 1 from saved outputs. The Figure 6 script performs a separate data-fission analysis and refits models. R is the primary language; selected comparisons use MATLAB, and compiled routines require a C++ toolchain.

Description

Code formats

[x] Script files: R, MATLAB, C and C++.

[x] Package: R package `treeFA`.

[] Reproducible report executing the analyses.

[x] Shell scripts: per-study SLURM launchers.

[] Other.

Supporting software requirements

Primary software

The supplied `treeFA` source package is version 0.0.2, as recorded in `code/core_code/treeFA/DESCRIPTION`. A saved historical analysis records R 4.2.3 on 64-bit Linux under Ubuntu 22.04.5 LTS. This record does not define a tested environment for every script in the package.

The runners for experiments 1-4 and 6 include MATLAB Relative Shift comparisons. Experiment 5, the eleven prediction analyses and the plotting/table scripts do not require MATLAB. The Figure 6 analysis compiles additional C++ code using `Rcpp::sourceCpp`. A complete MATLAB version, toolbox and compiler specification is not included in the current environment record.

Libraries and dependencies

The package imports `MASS` and `Rcpp`, links to `Rcpp` and `RcppEigen`, and suggests `glmnet` and `testthat` (at least 3.0.0). Analysis dependencies also include `Matrix`, `rare`, `mclust`, `pROC` and `readr`, depending on the study.

Figures 3 and 4 use `dplyr`, `tidyr`, `purrr`, `tibble`, `RColorBrewer`, `colorspace`, `igraph`, `ggplot2`, `ggplotify` and `patchwork`; their scripts check for `geom_boxplot(outliers = FALSE)` support. Figure 5 uses `ggplot2` and `patchwork`. Table 1 uses standard R packages, including `grid`.

Figure 6 additionally uses `glmnet`, `car`, `readr`, `dplyr`, `tidyr`, `igraph`, `tidygraph`, `ggraph`, `patchwork`, `ggplot2`, `Rcpp` and `RcppEigen`.

`manuscript/ACC/historical_environment.txt` records historical versions including `treeFA` 0.0.2, `rare` 0.1.1, `Matrix` 1.6-3, `glmnet` 4.1-8, `pROC` 1.18.5, `readr` 2.1.5 and `Rcpp` 1.0.13-1. The file is a saved session record, not a complete dependency lock file or evidence of a clean installation for this package version.

System and hardware requirements

A compatible C++ toolchain is needed to build `treeFA` and run Figure 6. The installed package under `code/core_code/rlib/` targets Linux and must be rebuilt for another platform. RS/SPG comparisons may also require platform-compatible MEX binaries. No GPU requirement is specified by the supplied workflow. The SLURM launchers expose project-root, module and resource settings; their wall-time limits are not measured runtimes.

Parallelization

The SLURM launchers distribute independent replicates through job arrays and request one CPU per task. The supplied records do not identify the number of concurrently allocated nodes or cores. The fitting scripts can also be invoked individually without a scheduler.

☐ No parallel code used.

☐ Multi-core parallelization on a single machine/node.

☐ Multi-machine/multi-node parallelization.

License

☐ MIT License.

☐ BSD.

☒ GPL v3.0, for the `treeFA` source package.

☐ Creative Commons.

☐ Other.

The GPL declaration in the package metadata applies to `treeFA`; it does not establish redistribution terms for third-party RS/SPG components or the datasets.

Part 3: Reproducibility workflow

Scope

The package provides the core estimation methods, six simulation studies and eleven real-data prediction analyses. The available result-generation scripts cover Figures 3-5, the computed columns of Table 1, and the Figure 6 data-fission analysis. The README identifies the exact entryptoints and outputs.

☐ Any numbers provided in text in the paper.

☒ The computational methods presented in the paper.

☐ All tables and figures in the paper.

☒ Selected tables and figures, with the limitations below.

Figure 3 uses experiment 1; Figure 4 uses experiment 2. Figure 5 panels (a,e), (b,f), (c,g) and (d,h) use experiments 3, 4, 5 and 6, respectively. Table 1 summarizes the eleven prediction analyses. Figure 6 uses the Sinha binary-outcome inputs and a separate model-fitting workflow.

Full manuscript coverage is not claimed. The drawing sources for methodological Figures 1 and 2 are not included. Table 1's signal-to-noise column is not calculated by the current script and is displayed as -.

Figure 6 code and its two data inputs are supplied, but its full refit and agreement with the paper have not been validated in this documentation update. Standalone supplementary studies are outside the declared main-text scope. Complete numerical agreement with the manuscript remains unverified.

Workflow

Location

☐ As part of the paper's supplementary material.

☐ In the final version of the Git repository.

☒ Other: the current local preparation package.

The intended repository is https://github.com/JinwenFu001/direct_feature_aggregation. No immutable identifier for the current package or supplementary/permanent archive is recorded in this version. The workflow described here refers to the local files, not to a verified matching public release.

Package structure

The package root contains `README.md` and four directories:

`code/`: core method implementations and shared helpers in `core_code/`, with execution scripts under `run code/simulations/` and `run code/real data study/`.

`data/`: processed real-data inputs, metadata and taxonomy.

`manuscript/`: figure PDFs, the Table 1 PDF and ACC documentation.

`output/`: saved simulation and real-data results, computed summaries and inference outputs.

The saved simulation results contain 200 replicates for each of experiments 1-5 and 400 for experiment 6. Each of the eleven prediction analyses has 200 saved replicates. These 3,600 `.RData` files allow the Figure 3-5 and Table 1 scripts to work without refitting the models.

Formats

☐ Single master code file.

☒ Wrapper shell scripts.

☐ Self-contained Quarto, R Markdown, Jupyter or other literate workflow.

☒ Text documentation of the workflow: the top-level `README.md`.

☐ Makefile.

☒ Other: individual R entrypoints for fitting, summaries and plotting.

Interpretation of saved results

Simulation objects retain method-specific results needed by their plotting scripts. Prediction-result files include fitted results, analysis configuration, training/test indices and session information. They are computational outputs, distinct from the input datasets under `data/`.

`Table 1.R` reads all 2,200 prediction files and saves `output/real data study/Table 1 summaries.rds`. Binary analyses have one summary row per replicate. For continuous outcomes, the script preserves the notebook's vector expansion before row binding and averaging; the result is not an equal-weight average of the 200 replicates. Missing-value propagation is retained. These rules and the uncomputed signal-to-noise column are material limitations when comparing the generated table with the manuscript.

Instructions

The top-level README gives setup details, all script paths and full replicate counts. Commands below run from the package root. Quoting is required because some paths contain spaces. The scripts locate their dependencies relative to the project root; external software executables and library locations can be configured for the target system.

1. Install the dependencies listed in the README and build `treeFA` into a library compatible with the target platform. For example:

```
export TREEFA_PROJECT_ROOT="$PWD"
export TREEFA_R_LIB="$PWD/code/core_code/rlib-local"
export R_LIBS_USER="$TREEFA_R_LIB"
mkdir -p "$TREEFA_R_LIB"
R CMD INSTALL --library="$TREEFA_R_LIB" code/core_code/treeFA
```

2. Generate the supported figures and Table 1 columns from saved results. These commands do not fit the models:

```
Rscript --vanilla "code/run code/simulations/experiment1/plot_figure3.R"
Rscript --vanilla "code/run code/simulations/experiment2/Figure 4.R"
Rscript --vanilla "code/run code/simulations/experiment3/Figure 5.R"
Rscript --vanilla "code/run code/real data study/Table 1.R"
```

The Figure 3 script currently writes `manuscript/figures/fig2joint.pdf`, while the prepared figure is named `Figure 3.pdf`. Figures 4 and 5 use the corresponding `Figure 4.pdf` and `Figure 5.pdf` names; Figure 5 also writes a PNG. The table script writes `manuscript/tables/Table 1.pdf` and its summary `.rds` object.

3. For fresh fits, use the per-experiment and per-analysis runners documented in the README. Simulation runners accept `TREEFA_UU`; a set `SLURM_ARRAY_TASK_ID` takes precedence. Experiments 1-5 use IDs 1-200 and experiment 6 uses IDs 1-400. Prediction runners require `SLURM_ARRAY_TASK_ID` in 1:200 and read inputs directly from `data/`. `TREEFA_RESULT_DIR` can direct fresh results to a separate location, preserving the saved reference files. SLURM launchers require the target cluster's project-root and module configuration.

4. Run Figure 6 separately after its R and C++ dependencies are available:

```
Rscript --vanilla \
  "code/run code/real data study/sinha_2016_binary/Figure 6.R"
```

This command reads `data/sinha_2016_data.RData` and `data/sinha_2016_genera.tsv`, compiles the supplied notebook solver and performs data fission, cross-validation and fitting. It writes `manuscript/figures/Figure 6.pdf`, `CRC_inference.csv` and `CRC_Fusobacterium_contrasts_BH.csv`; the CSVs are under `output/real data study/sinha_2016_binary/`. It does not use the 200 prediction replicates underlying Table 1.

Expected run-time

An end-to-end runtime on a specified system has not been measured for this package version. Reading saved results and rerunning all model fits are different workloads; method-level timing records and scheduler time limits do not establish the complete elapsed time. No runtime category is selected.

☐ Less than 1 minute.

☐ 1-10 minutes.

☐ 10-60 minutes.

☐ 1-8 hours.

☐ More than 8 hours.

☐ Not feasible on a desktop machine.

Additional information

Scripts specify random seeds for their stochastic calculations. Historical saved results and existing rendered figures do not, by themselves, establish a clean-environment rerun. Data documentation, complete software versioning, numerical agreement and archival availability remain limitations of this preparation version.